

PYTHON FUNDAMENTALS

Homework Assignment

Course:	420-N55(IOT)
Assignment	Python Programming Fundamentals
Total Marks	100
Submission	Six Python files + provided temperature data files

Due Date Sep 8 2026, 9:30 present to the teacher

Goal: Practice the main Python concepts covered in class and combine them in a realistic three-day home temperature analysis.

General Instructions

- Complete all six parts.
- Use meaningful variable and function names.
- Add short comments where they improve readability.
- Test your program before submission.
- For Part 6, use the three text files provided with the assignment.
- Do not manually type the temperature records into your Python program.
- Do not hard-code the final answers in Part 6.
- You can use your virtual python environment if you need to install any library or package

Part	Topic	Marks
1	Functions	10
2	Loops	15
3	Lists, Tuples & Dictionaries	15
4	Files & Exception Handling	15
5	*args & **kwargs	15
6	Temperature Case Study	30

PART 1 — Functions in Python

10 marks

Question 1 — Temperature Conversion

Create a function named:

```
celsius_to_fahrenheit(celsius)
```

The function receives a temperature in Celsius and returns its Fahrenheit equivalent. Test your function with at least three different temperatures.

Question 2 — Find the Largest Number

Create a function named:

```
find_largest(a, b, c)
```

The function receives three numbers and returns the largest number.

```
find_largest(12, 8, 20)
# Expected result: 20
```

Question 3 — Pass or Fail

Create a function named:

```
check_grade(mark)
```

- 60 or higher → "Pass"
- Below 60 → "Fail"

Ask the user for a mark and call the function to display the result.

PART 2 — Loops in Python**15 marks****Question 1 — Counting**

Using a for loop, display the numbers from 1 to 20.

Question 2 — Even Numbers

Using a loop, display only the even numbers between 1 and 50.

```
2
4
6
8
...
```

Question 3 — Calculate a Total and Average

Use the following list:

```
numbers = [12, 18, 25, 31, 42, 50]
```

1. Calculate the total.
2. Count how many values are in the list.
3. Calculate the average.
4. Do not use sum() to calculate the total.

PART 3 — Python Data Structures**15 marks**

Work with lists, tuples, and dictionaries.

A. List

```
devices = ["Laptop", "Router", "Printer", "Server", "Switch"]
```

5. Display every device using a loop.
6. Add "Tablet" to the list.
7. Remove "Printer".
8. Display the number of devices in the final list.

B. Tuple

```
computer = ("Dell", "Laptop", 16, 512)
```

The values represent Brand, Device Type, RAM in GB, and Storage in GB. Display each piece of information with a meaningful message.

```
Brand: Dell
RAM: 16 GB
```

Then answer in a Python comment:

```
# Why might a tuple be useful when we do not want data to change?
```

C. Dictionary

```
student = {
    "name": "Alex",
    "student_id": 10025,
    "program": "IT",
    "semester": 1
}
```

9. Display the student's name.
10. Display the program.
11. Change the semester from 1 to 2.
12. Add a new key named "email".
13. Use a loop to display all keys and values.

PART 4 — Files and Exception Handling

15 marks

Create a Python program that asks the user to enter three numbers. Store the three numbers in a text file named:

```
numbers.txt
```

Each number should be written on a separate line. Then reopen the file and calculate the average.

Required exception handling: Your program must handle at least `ValueError` and `FileNotFoundError`.

```
try:
    ...
except ValueError:
    ...
except FileNotFoundError:
    ...
```

If the user enters invalid data, display a useful message instead of allowing the program to crash.

```
Invalid input. Please enter a number.
```

PART 5 — *args and **kwargs

15 marks

A. *args

Create a function named:

```
calculate_average(*args)
```

The function should accept any number of numeric arguments and return their average.

```
calculate_average(10, 20, 30)
# Result: 20.0

calculate_average(10, 20, 30, 40, 50)
```

B. **kwargs

Create a function named:

```
display_device(**kwargs)
```

It should accept different information about a computer device and use a loop to display every key and value.

```
display_device(
    brand="Dell",
    model="Latitude",
    ram="16 GB",
    storage="512 GB"
)
```

Then write one Python comment explaining the difference between *args and **kwargs.

PART 6 — Case Study: Home Temperature Monitoring

30 marks

You have installed three temperature sensors in your home. Each room represents one sensor:

- Room 101
- Room 102
- Room 103

The sensors recorded temperature measurements for three consecutive days. Use the three files provided with this assignment:

```
my_home_temperature-2026-09-01.txt
my_home_temperature-2026-09-02.txt
my_home_temperature-2026-09-03.txt
```

Field	Meaning
id	Unique measurement ID
datetime	Date and time of measurement
room_number	Sensor / room number
temperature_c	Temperature in °C

Each file begins with this header:

```
id,datetime,room_number,temperature_c
```

Example records:

```
1,2026-09-01 06:00,101,18.4
2,2026-09-01 06:00,102,19.2
3,2026-09-01 06:00,103,18.0
```

Important: The IDs do not restart every day. Day 1 uses IDs 1–24, Day 2 continues with IDs 25–48, and Day 3 continues with IDs 49–72.

Your Task

1. Read all three files

Store the three filenames in a list and use a loop to read every file. Do not copy the temperature records into your Python program.

2. Handle the file header

Do not treat the header line as a sensor measurement.

3. Combine the data

Store all records from all three files together using lists, dictionaries, tuples, or a combination.

4. Use correct data types

Convert id and room_number to integers and temperature_c to a float.

5. Order by ID

Sort all measurements by id so the final data runs from 1 through 72 in order.

6. Find the hottest measurement

Find the highest temperature in the complete three-day dataset and display its temperature, date, time, room number, and measurement ID.

7. Calculate the average for each sensor

Calculate the three-day average temperature separately for Room 101, Room 102, and Room 103.

8. Calculate the overall average

Calculate the average temperature of all 72 measurements.

9. Use exception handling

Handle at least FileNotFoundError and ValueError with meaningful messages.

10. Use functions

Part 6 must contain at least three functions. You may create functions for reading files, calculating averages, finding the maximum temperature, or displaying results.

Expected output style: Your exact formatting may be different, but your program must clearly report the hottest reading, the three sensor averages, and the overall average.

HOTTEST READING

Temperature: XX.X °C

Date: YYYY-MM-DD

Time: HH:MM

Room: XXX

ID: XX

AVERAGE SENSOR TEMPERATURE

Room 101: XX.XX °C

Room 102: XX.XX °C

Room 103: XX.XX °C

Overall Average: XX.XX °C

Suggested Program Flow

```
START
↓
Create list of 3 filenames
↓
Read File 1 → Read File 2 → Read File 3
↓
Combine all records
↓
Sort records by ID
↓
Find hottest measurement
↓
Calculate average for each room/sensor
↓
Calculate overall average
↓
Display results
↓
END
```

Part 6 Criterion	Marks
Read all three files correctly	4
Data conversion and header handling	3
Combine and sort records by ID	3
Find hottest measurement correctly	4
Three sensor averages	4
Overall average	2
Functions	3
Loops and data structures	2
Exception handling	3
Readability and comments	2

Submission Requirements

Submit the following Python files:

```
lastname_firstname_part1.py  
lastname_firstname_part2.py  
lastname_firstname_part3.py  
lastname_firstname_part4.py  
lastname_firstname_part5.py  
lastname_firstname_part6.py
```

Part 6 rule: Your program must work with the original three temperature files supplied by the instructor without changing their contents.

Final Checklist

- ☐ All six parts are completed.
- ☐ Programs run without syntax errors.
- ☐ Part 6 reads the files instead of hard-coding the data.
- ☐ Part 6 sorts measurements by ID.
- ☐ Part 6 reports the hottest reading and all required averages.
- ☐ Exception handling is included where required.
- ☐ Files use the requested naming format.